

integration_sec05

5. Phase 4: Model Discovery and Registry Integration

The discovery and registry subsystem forms the informational backbone of the LLMsVerifier integration, responsible for identifying available language models across all configured providers, maintaining a durable and queryable catalog of verified models, and enforcing admission policies that prevent unqualified models from entering the translation pipeline. This phase implements a resilient three-tier discovery architecture that balances user control against automation, a local SQLite-backed registry with full-text search capabilities, and a gatekeeping layer that applies composite-score and health-status filters before any model can be selected for inference. All discovery results are persisted to the unified registry, indexed for efficient lookup, and exposed through a stable REST API that serves both the internal scorer and external administrative tools. The subsystem is designed to operate continuously in the background, refreshing model availability at configurable intervals while broadcasting change events to interested consumers throughout the HelixTranslate system.

5.1 Three-Tier Discovery Service

The `DiscoveryService` orchestrates model enumeration across three independent tiers, each offering a different trade-off between authority and freshness. Tier 1 draws from the user's explicit configuration in `configs/verifier.yaml`, where administrators can whitelist specific model IDs, set per-model score thresholds, or pin provider preferences. This tier carries the highest priority because user-specified values represent deliberate policy decisions that must override any dynamically discovered alternative. When Tier 1 returns at least one model, the discovery process terminates immediately and returns only the user-configured entries, ensuring that no provider API or community registry can supersede administrator intent.

Tier 2 performs dynamic enumeration by querying each configured provider's model list endpoint. OpenAI-compatible providers expose `/v1/models`; Anthropic provides `v1/models`; and custom adapters normalize their native APIs to the same response shape. Each provider query is executed with a 30-second timeout and runs in its own goroutine, with results aggregated through a buffered channel. The provider API tier is the primary source of truth for newly released models or pricing changes, but it is only consulted when the user configuration tier returns no entries. This fallback ordering prevents API quota consumption when the administrator has already specified a closed set of acceptable models.

Tier 3 queries the `models.dev` community registry, a publicly maintained database of language model metadata including community-verified scores, pricing benchmarks, and capability tags. This tier serves as the final fallback when both user configuration and live provider APIs are unavailable or return empty results. The registry endpoint responds within 10 seconds and provides models that may not yet be present in any configured provider account, enabling the system to pre-evaluate models before they are commercially available. Because community data can be stale or unverified, Tier 3 results receive the lowest priority and are always marked with `source: "community"` for downstream auditing.

The cascading logic is implemented in `DiscoverAll`, which sequentially attempts each tier, deduplicates results by model ID, and returns the first non-empty set. Deduplication preserves the highest-priority entry when the same model appears in multiple tiers, ensuring that a user-configured whitelist entry always dominates a matching API-discovered entry even if both tiers were eventually queried.

Table 5.1: Discovery Tier Configuration

Tier	Source	Timeout	Priority	Refresh
1	User config (<code>configs/verifier.yaml</code>)	5s	Highest	On config change
2	Provider API dynamic enumeration	30s/provider	Medium	1 hour
3	<code>models.dev</code> community registry	10s	Lowest (fallback)	24 hours

The `DiscoveryService` struct embeds three tier-specific implementations, each satisfying a common `Discovery` interface that abstracts the underlying enumeration mechanism. A `sync.RWMutex` protects the service state during concurrent discovery runs, ensuring that background sync goroutines and on-demand API calls do not race on shared configuration.

```
// pkg/discovery/service.go
type DiscoveryService struct {
    tier1 *ConfigDiscovery // User-configured
    tier2 *APIDiscovery   // Provider APIs
    tier3 *RegistryDiscovery // models.dev
    mu    sync.RWMutex
}

func (s *DiscoveryService) DiscoverAll(ctx context.Context)
    ([]DiscoveredModel, error) {
    var allModels []DiscoveredModel

    // Tier 1: User config
    if models, err := s.tier1.Discover(ctx); err == nil {
        allModels = append(allModels, models...)
    }
    if len(allModels) > 0 {
        return deduplicate(allModels), nil
    }

    // Tier 2: Provider APIs
```

```

    if models, err := s.tier2.Discover(ctx); err == nil {
        allModels = append(allModels, models...)
    }
    if len(allModels) > 0 {
        return deduplicate(allModels), nil
    }

    // Tier 3: models.dev
    if models, err := s.tier3.Discover(ctx); err == nil {
        allModels = append(allModels, models...)
    }
    return deduplicate(allModels), nil
}

```

The deduplicate helper selects entries by priority order. Each `DiscoveredModel` carries a `SourceTier` field populated by the originating discovery implementation. When two entries share the same `ModelID`, the one with the numerically lower tier value is retained. After deduplication, the final slice is sorted lexicographically by model ID to provide deterministic output across repeated calls. This deterministic ordering is critical for cache key generation and for test assertions that verify discovery behavior.

Each tier implementation handles its own caching and error semantics. `ConfigDiscovery` watches `configs/verifier.yaml` via `fsnotify` and reloads on `Write` events, providing near-instant refresh without polling. `APIDiscovery` maintains an in-memory cache of provider responses with a 1-hour TTL, serving stale data during transient provider outages while queuing a background refresh. `RegistryDiscovery` implements exponential backoff for failed requests to the `models.dev` endpoint, waiting 1 minute after the first failure, 5 minutes after the second, and capping at 30 minutes to avoid hammering the community service. All three implementations emit structured log entries at `INFO` level for successful discoveries and `WARN` level for timeouts or non-2xx HTTP responses.

5.2 Unified Model Registry

Discovered models are persisted to a local SQLite database that serves as the single source of truth for all downstream components. The registry schema is designed to capture every dimension needed for model selection: identification, verification status, scoring, capabilities, pricing, availability, and temporal metadata. The `verified_models` table enforces data integrity through `CHECK` constraints on enumerated columns and uses a composite index strategy optimized for the query patterns observed in production.

The `model_id` column stores the canonical model identifier as returned by the provider (e.g., `gpt-4o-2024-08-06`, `claude-sonnet-4-20250514`). The `provider_id` foreign key links to a separate `providers` table defined in Phase 3. The `status` column implements the model lifecycle: `pending` for newly discovered models awaiting first evaluation, `verified` for models that have completed at least one full scoring run, `failed` for models that failed verification due to repeated inference errors or policy violations, and `retired` for models that have been removed from all provider APIs and community registries. A model in `retired` status remains in the database for audit purposes but is excluded from all selection queries.

The `composite_score` column stores the weighted aggregate score produced by Phase 6's scoring pipeline, ranging from 0.0 to 10.0. The `tier` column classifies models into four bands: premium (score ≥ 8.0), standard (score ≥ 6.0), budget (score ≥ 4.0), and unranked (below 4.0 or no score). Both `capabilities` and `pricing` columns store JSON documents, allowing the schema to evolve without migration scripts as providers introduce new features or pricing dimensions. The `is_available` boolean reflects the most recent health check result and is used as a fast filter to exclude down providers without evaluating score predicates.

Table 5.2: SQLite Registry Schema

```
-- pkg/registry/schema.sql
CREATE TABLE verified_models (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  model_id TEXT UNIQUE NOT NULL,
  provider_id TEXT NOT NULL,
  name TEXT NOT NULL,
  status TEXT NOT NULL CHECK(status IN
    ('pending', 'verified', 'failed', 'retired')),
  composite_score REAL,
  tier TEXT CHECK(tier IN
    ('premium', 'standard', 'budget', 'unranked')),
  capabilities TEXT, -- JSON
  pricing TEXT, -- JSON
  verified_at TIMESTAMP,
  last_checked TIMESTAMP,
  is_available BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_models_score ON verified_models(composite_score);
CREATE INDEX idx_models_provider ON verified_models(provider_id);
CREATE INDEX idx_models_status ON verified_models(status);
CREATE INDEX idx_models_available
  ON verified_models(is_available, composite_score);
```

The index design reflects the system's dominant access patterns. `idx_models_score` accelerates `FilterByScore` queries used by the gatekeeper to find all models above a threshold. `idx_models_provider` speeds up provider-specific administrative views. `idx_models_status` supports batch operations that target models in a particular lifecycle state, such as re-evaluating all failed models after a configuration change. The composite index `idx_models_available` is the most critical for runtime performance: nearly every production query includes `is_available = TRUE`, and the secondary ordering by `composite_score` allows the query planner to satisfy `ORDER BY composite_score DESC` without an additional sort step.

The `ModelRegistry` type provides the Go API for all database operations. It uses `database/sql` with `modernc.org/sqlite` as the driver, avoiding CGO dependencies for simpler cross-compilation. Connection pooling is configured with a maximum of 10 open connections and 5 idle connections, sufficient for the single-node deployment model. All methods accept a `context.Context` for timeout and cancellation propagation, and queries exceeding 5 seconds are logged at `WARN` level for operational monitoring.

```
// pkg/registry/registry.go
type ModelRegistry struct {
    db *sql.DB
}

func (r *ModelRegistry) ListModels(ctx context.Context, filter
    ModelFilter) ([]Model, error)
func (r *ModelRegistry) GetModel(ctx context.Context, modelID
    string) (*Model, error)
func (r *ModelRegistry) FilterByScore(ctx context.Context,
    minScore float64) ([]Model, error)
func (r *ModelRegistry) FilterByProvider(ctx context.Context,
    providerID string) ([]Model, error)
func (r *ModelRegistry) SearchModels(ctx context.Context, query
    string) ([]Model, error)
```

ListModels accepts a ModelFilter struct containing optional fields for ProviderID, Status, MinScore, MaxScore, Tier, and IsAvailable. The implementation builds parameterized SQL dynamically, including only non-zero filter predicates in the WHERE clause. Results are ordered by composite_score DESC when a score filter is present, otherwise by name ASC. GetModel performs a primary-key lookup by model_id and returns sql.ErrNoRows as a sentinel when the model is not found, allowing callers to distinguish between missing data and database errors. FilterByScore and FilterByProvider are convenience methods that delegate to ListModels with appropriately populated filters, exposed separately because they represent the two most common query patterns in the selection pipeline.

SearchModels implements full-text search over the name column using SQLite's LIKE operator with % wildcards prepended and appended to the query term. While this is not a true full-text search engine, it satisfies the administrative use case of finding models by partial name match (e.g., searching "gpt-4" to find all GPT-4 variants). For installations requiring more sophisticated search, the method can be replaced with an FTS5-backed implementation without changing the interface. Search results are ordered by composite_score DESC to surface the highest-quality matches first.

Table 5.3: Registry Query API Reference

Method	Parameters	Return	SQL Complexity	Cache
ListModels	ModelFilter	[]Model	WHERE + ORDER	5min
GetModel	modelID	*Model	PRIMARY KEY	10min
FilterByScore	minScore	[]Model	WHERE score >=	None
FilterByProvider	providerID	[]Model	WHERE provider	1min
SearchModels	query	[]Model	LIKE on name	None

The registry implements a read-through caching layer using an in-memory LRU cache with a default capacity of 1,000 entries. GetModel results are cached for 10 minutes because individual model metadata changes infrequently. ListModels with filters is cached for 5 minutes, with cache keys computed from the serialized filter struct. Score and provider filters bypass the cache because they are typically invoked by automated pipelines that benefit from fresh data and operate at frequencies where cache hits would be rare. Search queries are also uncached due to the high

cardinality of possible query strings. Cache invalidation is triggered by `ModelUpdatedEvent` broadcasts (see Section 5.4), ensuring that model metadata changes are reflected in subsequent queries without waiting for TTL expiration.

5.3 Verified Model Gatekeeping

The gatekeeping layer enforces admission control, determining whether a specific model is eligible for translation work based on its verification status, composite score, and health state. This component sits at the boundary between model discovery and model selection: it answers the binary question "can this model be used right now?" with a detailed reason when the answer is negative. The `Gatekeeper` type is initialized with a `minScoreThreshold` (default 5.5) and a reference to the `ModelRegistry` for lookups.

The `IsModelAvailable` method performs a synchronous database lookup followed by a sequence of predicate checks. First, it verifies that the model exists in the registry; a missing model returns `false` with reason `"not_found"`. Second, it checks that the model's status column equals `"verified"`; models in pending, failed, or retired states are rejected with reason `"verification"`. Third, it compares the model's `CompositeScore` against the configured threshold; models scoring below the minimum are rejected with reason `"score"`. Fourth, it examines the `HealthStatus` field, which must be either `"healthy"` or `"degraded"`; models marked `"down"` or `"unknown"` are rejected with reason `"health"`. Only when all four predicates pass does the method return `true` with an empty reason string.

```
// pkg/gatekeeper/gatekeeper.go
type Gatekeeper struct {
    minScoreThreshold float64
    registry           *ModelRegistry
}

func (g *Gatekeeper) IsModelAvailable(modelID string) (bool,
    string, error) {
    model, err := g.registry.GetModel(context.Background(),
        modelID)
    if err != nil {
        return false, "not_found", err
    }
    if model.Status != "verified" {
        return false, "verification", nil
    }
    if model.CompositeScore < g.minScoreThreshold {
        return false, "score", nil
    }
    if model.HealthStatus != "healthy" && model.HealthStatus !=
        "degraded" {
        return false, "health", nil
    }
    return true, "", nil
}
```

The gatekeeper is intentionally conservative: a model must demonstrate sustained positive evidence to pass. This design prevents transiently available models with unknown quality from entering the translation pipeline. The `minScoreThreshold` is configurable at startup via the `VERIFIER_MIN_SCORE` environment variable and can be adjusted at runtime through a configuration hot-reload endpoint. Lowering the threshold increases model diversity but may reduce average translation quality; raising it improves quality guarantees but decreases redundancy in the model pool.

The reason strings returned by `IsModelAvailable` are used for observability and debugging. Each rejection reason increments a Prometheus counter `gatekeeper_rejections_total{reason="..."}`, allowing operators to monitor whether rejections are dominated by score failures (indicating the threshold may be too high), health failures (indicating provider instability), or verification failures (indicating models stuck in pending state). These metrics feed into the alerting rules defined in Phase 8.

Table 5.4: Gatekeeping Decision Matrix

Verification	Score $\geq$ 5.5	Health	Available	Reason
Pass	Yes	Healthy	TRUE	OK
Pass	Yes	Degraded	TRUE	DEGRADED
Pass	Yes	Down	FALSE	PROVIDER_DOWN
Pass	No	Any	FALSE	SCORE_LOW
Fail	Any	Any	FALSE	VERIFICATION_FAILED
Pending	Any	Any	FALSE	PENDING

When a model passes gatekeeping with DEGRADED health status, it is admitted to the pool but with reduced traffic weight. The selection algorithm (Phase 6) applies a 0.5x multiplier to the degraded model's routing probability, shifting load toward fully healthy alternatives while maintaining the degraded model as a warm standby. If all models in a tier are degraded, the system continues operating at reduced capacity rather than rejecting requests, providing graceful degradation rather than hard failure. A model marked Down is completely excluded from the selection set, and if all models across all providers are down, the system returns a 503 Service Unavailable with a descriptive error message listing the affected providers.

5.4 Background Discovery and Event Broadcasting

Discovery is not a one-time operation. Provider catalogs change as new models are released, old models are deprecated, and pricing is adjusted. The `DiscoveryService` runs a background synchronization loop that periodically re-executes the three-tier discovery pipeline, compares results against the current registry state, and publishes change events for any detected differences.

The `StartBackgroundSync` method accepts a context and a refresh interval (default 1 hour). It creates a `time.Ticker` and adds a random jitter of up to 10% of the interval duration before the first tick. This jitter prevents thundering herd behavior when multiple `HelixTranslate` instances start simultaneously and share the same configuration. Each tick triggers a full `DiscoverAll` call; when new models are

found, an event is published to the central event bus for each discovered entry. Errors during discovery are logged but do not terminate the goroutine, ensuring that transient network issues do not permanently disable background synchronization.

```
// pkg/discovery/background.go
func (s *DiscoveryService) StartBackgroundSync(ctx
    context.Context, interval time.Duration) {
    ticker := time.NewTicker(interval)
    jitter := time.Duration(rand.Int63n(int64(interval / 10)))
    go func() {
        time.Sleep(jitter) // Prevent thundering herd
        for {
            select {
            case <-ticker.C:
                models, err := s.DiscoverAll(ctx)
                if err != nil {
                    log.Printf("discovery error: %v", err)
                    continue
                }
                for _, m := range models {
                    events.Publish(ctx, events.ModelDiscovered{
                        ModelID:      m.ID,
                        ProviderID: m.ProviderID,
                    })
                }
            case <-ctx.Done():
                ticker.Stop()
                return
            }
        }
    }()
}
```

The event bus (pkg/events) provides a publish-subscribe mechanism with at-least-once delivery semantics. Subscribers register typed handlers for specific event kinds, and the bus dispatches published events to all matching handlers in parallel. Event delivery is asynchronous, with a 30-second timeout per handler to prevent slow consumers from blocking the publisher. Events that fail delivery are retried up to 3 times with exponential backoff, then written to a dead-letter log for manual inspection.

Four event types capture the full lifecycle of a model in the registry. The `ModelDiscoveredEvent` fires when a background sync or manual discovery finds a model not currently present in the registry. The `ModelUpdatedEvent` fires when an existing model's metadata changes, such as a pricing update or capability addition detected during a subsequent provider API query. The `Changes` map contains only the fields that differ from the previous registry entry, allowing subscribers to perform incremental updates. The `ModelScoreChangedEvent` fires when a model's composite score crosses a tier boundary, enabling subscribers to react to promotions (e.g., budget to standard) or demotions (premium to standard) with appropriate log entries or alerts. The `ModelRemovedEvent` fires when a model that was previously in the registry disappears from all discovery tiers for three consecutive sync cycles, indicating provider deprecation or retirement.

```
// pkg/events/model_events.go
type ModelDiscoveredEvent struct {
    ModelID    string
    ProviderID string
    Name       string
    Source     string
    Timestamp  time.Time
}

type ModelUpdatedEvent struct {
    ModelID    string
    Changes    map[string]interface{}
    Timestamp  time.Time
}

type ModelScoreChangedEvent struct {
    ModelID    string
    OldScore   float64
    NewScore   float64
    OldTier    string
    NewTier    string
    Timestamp  time.Time
}

type ModelRemovedEvent struct {
    ModelID    string
    Reason     string
    Timestamp  time.Time
}
```

The registry subscribes to all four event types and updates its SQLite database accordingly. On `ModelDiscoveredEvent`, it inserts a new row with `status = 'pending'` and queues an asynchronous scoring job. On `ModelUpdatedEvent`, it applies the changes map using a JSON merge patch for the capabilities and pricing columns, and bumps the `updated_at` timestamp. On `ModelScoreChangedEvent`, it updates both the `composite_score` and `tier` columns, triggering any downstream cache invalidation. On `ModelRemovedEvent`, it transitions the model to `status = 'retired'` rather than deleting the row, preserving historical audit data. The scoring pipeline (Phase 6) subscribes to `ModelDiscoveredEvent` to trigger initial evaluations and to `ModelScoreChangedEvent` to log tier transitions for analytics.

Administrators can trigger on-demand discovery through the REST API (Section 5.5), which bypasses the background ticker and immediately executes `DiscoverAll`. This is useful when a new model has just been enabled in a provider account and the operator does not want to wait for the next scheduled sync. On-demand discovery publishes the same events as background sync, ensuring consistent handling regardless of trigger source.

5.5 REST API Endpoints

The discovery and registry functionality is exposed through a JSON REST API under the `/api/v1/models` path prefix. All endpoints require authentication via the bearer token middleware established in Phase 3, and all responses include standard `X-Request-ID` headers for traceability. The API follows REST conventions: collection endpoints return arrays, singleton endpoints return objects, and write operations return the affected resource. Error responses conform to RFC 7807 Problem Details format with `application/problem+json` content type.

`GET /api/v1/models` returns a paginated list of models from the registry, accepting query parameters for `provider`, `status`, `min_score`, `tier`, and `available` to filter results. Pagination uses cursor-based navigation with a `limit` parameter (default 50, max 200) and an `after` cursor for consistent ordering across insertions. The response includes a `Link` header with `next` and `prev` relations when additional pages exist. Each model object in the response array contains the full registry record with capabilities and pricing JSON objects inlined for convenience.

`GET /api/v1/models/{id}` returns a single model by its canonical model ID. If the model is not found, the endpoint returns 404 Not Found with a problem detail indicating "model not found". If the model exists but is in retired status, it is returned with a 200 OK and an additional `retired: true` field in the response body, allowing clients to distinguish retired models from active ones.

`POST /api/v1/models/discover` triggers an on-demand discovery run across all three tiers. The endpoint returns 202 Accepted immediately with a JSON body containing a `discovery_id` UUID. The actual discovery executes asynchronously, and clients can poll `GET /api/v1/models/discover/{discovery_id}` for status updates. The status endpoint returns `pending`, `running`, `completed`, or `failed`, and on completion includes a summary of models discovered, updated, and removed during the run. This asynchronous pattern prevents long-running discovery operations from occupying HTTP connections and timing out at load balancer or client levels.

`POST /api/v1/models/refresh` forces a refresh of model metadata from provider APIs for all models currently in the registry with `status = 'verified'`. Unlike `discover`, which looks for new models, `refresh` updates existing records with the latest provider data. This endpoint is useful after provider maintenance windows or pricing changes. It also returns 202 Accepted and follows the same async polling pattern as `discover`.

```
// pkg/api/handler.go
func (h *Handler) ListModels(w http.ResponseWriter, r
    *http.Request) // GET /api/v1/models
func (h *Handler) GetModel(w http.ResponseWriter, r
    *http.Request) // GET /api/v1/models/{id}
func (h *Handler) TriggerDiscovery(w http.ResponseWriter, r
    *http.Request) // POST /api/v1/models/discover
func (h *Handler) RefreshModels(w http.ResponseWriter, r
    *http.Request) // POST /api/v1/models/refresh
```

The handler implementation uses a shared `ModelRegistry` instance initialized at application startup and injected through the handler's constructor. This dependency injection pattern ensures that API handlers and background discovery processes

operate on the same database connection pool and cache layer, maintaining consistency without explicit synchronization. Request parsing validates all query parameters and returns 400 Bad Request with detailed validation errors for invalid filter combinations (e.g., `min_score` above the maximum possible value of 10.0, or a `tier` value not in the allowed enumeration).

Response serialization uses the same JSON encoder configuration as the rest of the HelixTranslate API: indented output in development mode, compact output in production, and HTML-escaping disabled to properly render Unicode characters in model names. The capabilities and pricing JSON columns are unmarshaled into `map[string]interface{}` during serialization so they appear as nested objects in the response rather than escaped JSON strings. Database timestamps are formatted as RFC 3339 strings in the response body, with null timestamps for models that have never been verified or checked omitted entirely using `omitempty` struct tags.

The API layer implements request logging middleware that records method, path, query parameters, response status, and duration for every request. These logs are emitted at `INFO` level for 2xx responses, `WARN` level for 4xx responses, and `ERROR` level for 5xx responses. Aggregate metrics are exported via Prometheus counters `api_requests_total` and histograms `api_request_duration_seconds`, labeled by method, endpoint pattern, and status code. Together, these observability features provide operators with full visibility into discovery and registry API usage patterns and performance characteristics.